INFRASTRUCTURE / PORTFOLIO CASE STUDY

Automated Backup System

Scheduled, verified, multi-tier backups for a fleet of application databases. Cron-driven Bash scripts dump every PostgreSQL and MySQL database, checksum each artifact, replicate it to a local storage server with rsync, push it offsite to AWS S3 under a lifecycle policy, prune by retention rules, and raise a Slack alert the moment anything fails.

Bash	**Cron**	**rsync**	**AWS S3**
PostgreSQL	**MySQL**	**SHA-256 integrity**	**Slack alerts**

Environment
Northstar Animation Studios — Linux pipeline

Sources
PostgreSQL + MySQL across app servers

Destinations
Local storage server + AWS S3 (offsite)

Summary. Several studio applications each own a database — Ansible AWX on PostgreSQL, the internal wiki and dashboards on MySQL — and losing any of them means lost production work. This project automates their protection end to end. A scheduled Bash routine dumps each database, compresses it, and writes a SHA-256 checksum so corruption is detectable. Each artifact is then replicated to an on-site storage server with rsync and pushed offsite to AWS S3, where a lifecycle policy transitions old backups to cheaper cold storage and eventually expires them. Local copies are pruned by a retention window, restores are periodically test-verified, and any failure in the chain triggers an immediate Slack notification. The design follows the 3-2-1 rule: three copies, on two kinds of media, with one copy off-site.

1 — Strategy & Architecture

The system is built around the 3-2-1 backup rule. Every database produces a primary dump on its own host, which is replicated to a separate on-site storage server and then copied off-site to AWS S3. A single lost disk, a corrupted dump, or even the loss of the whole server room never takes out every copy.

The 3-2-1 layout

- **Copy 1 — source host.** The fresh dump is written locally on the application server, kept for a short window for fast restores.
- **Copy 2 — storage server.** rsync replicates dumps to a dedicated on-site backup server on different hardware.
- **Copy 3 — AWS S3 (off-site).** Dumps are pushed to S3 and aged through a lifecycle policy into cold storage, protecting against site-level loss.

What gets backed up

Application	Engine	Source host	Schedule
Ansible AWX	PostgreSQL	`ns-awx01`	Daily 02:00
Metrics / dashboards	MySQL	`ns-grafana01`	Daily 02:15
Internal wiki	MySQL	`ns-wiki01`	Daily 02:30
Asset / job tracker	PostgreSQL	`ns-tracker01`	Daily 02:45
Container registry meta	PostgreSQL	`ns-registry01`	Weekly Sun 03:00

Pipeline flow

1	**Dump**	pg_dump / mysqldump produces a consistent, compressed dump per database.
2	**Checksum**	A SHA-256 hash is written alongside each artifact for later verification.
3	**Replicate**	rsync mirrors the day's dumps to the on-site storage server.
4	**Off-site**	aws s3 sync pushes the dumps to an S3 bucket with versioning enabled.
5	**Age & expire**	An S3 lifecycle policy moves old backups to Glacier and expires them on schedule.
6	**Prune**	Local and storage-server copies older than the retention window are deleted.
7	**Verify / alert**	Restores are periodically test-checked; any failure fires a Slack alert.

2 — Database Dumps

Each engine has a native, point-in-time-consistent dump tool. Credentials are never placed on the command line; they live in protected, per-engine credential files owned by the backup user.

2.1 — PostgreSQL

`pg_dump` in custom format (`-Fc`) is compressed and supports selective, parallel restore. The password is read from a `~/.pgpass` file (mode 0600), so nothing sensitive appears in process listings.

```
# ~/.pgpass (chmod 600) host:port:db:user:password
10.20.0.11:5432:awxdb:backup:REDACTED

# custom-format, compressed dump
pg_dump -Fc -h 10.20.0.11 -U backup -d awxdb \
        -f "$DEST/awxdb_$(date +%F).dump"
```

2.2 — MySQL

`mysqldump` with `--single-transaction` takes a consistent snapshot of InnoDB tables without locking the application out. Credentials come from a `--defaults-extra-file` rather than the command line.

```
# /opt/backup/mysql.cnf (chmod 600)
[client]
user=backup
password=REDACTED

# consistent dump, gzip-compressed on the fly
mysqldump --defaults-extra-file=/opt/backup/mysql.cnf \
          --single-transaction --quick wikidb \
  | gzip > "$DEST/wikidb_$(date +%F).sql.gz"
```

2.3 — Integrity checksums

Immediately after each dump, a SHA-256 checksum is recorded. This is what makes a backup *trustworthy* rather than merely present — a silently corrupted dump is caught at verification time instead of during a restore emergency.

```
# write a checksum next to each artifact
sha256sum "$DEST/awxdb_$(date +%F).dump" \
  > "$DEST/awxdb_$(date +%F).dump.sha256"

# verify (used before replication and during restore tests)
sha256sum -c "$DEST/awxdb_$(date +%F).dump.sha256"
```

3 — The Backup Script

A single hardened Bash script ties the steps together. It fails fast (`set -euo pipefail`), traps any error to a Slack notifier, and logs every run. The same script handles both engines, selected per host.

```
#!/usr/bin/env bash
# /opt/backup/db-backup.sh
set -euo pipefail

DEST="/srv/backups/$(hostname)"
DATE="$(date +%F)"
SLACK_WEBHOOK="https://hooks.slack.com/services/REDACTED"
mkdir -p "$DEST"

notify_fail() {
  curl -sf -X POST -H 'Content-type: application/json' \
    --data "{\"text\":\":x: Backup FAILED on $(hostname) — $1\"}" \
    "$SLACK_WEBHOOK" || true
}
trap 'notify_fail "step near line $LINENO"' ERR

# 1. dump (PostgreSQL example)
pg_dump -Fc -h 10.20.0.11 -U backup -d awxdb -f "$DEST/awxdb_$DATE.dump"

# 2. checksum + verify
sha256sum "$DEST/awxdb_$DATE.dump" > "$DEST/awxdb_$DATE.dump.sha256"
sha256sum -c "$DEST/awxdb_$DATE.dump.sha256"

# 3. replicate on-site, then off-site
rsync -az --delete "$DEST/" backup@10.20.0.70:/srv/backups/$(hostname)/
aws s3 sync "$DEST/" "s3://northstar-backups/$(hostname)/" --only-show-errors

# 4. prune local copies older than the retention window
find "$DEST" -type f -mtime +14 -delete

echo "backup OK: $(hostname) $DATE"
```

Because the error trap calls the Slack notifier, a failure at *any* step — a dump error, a checksum mismatch, an unreachable storage server, or an S3 permission problem — produces an alert rather than a silent gap in coverage.

4 — Off-site Replication & S3 Lifecycle

4.1 — rsync to the storage server

rsync transfers only changed blocks over SSH, so daily replication is fast and bandwidth-light even as the backup set grows. The storage server is a distinct machine, satisfying the ‘two media’ part of 3-2-1.

```
rsync -az --delete \
  /srv/backups/ns-awx01/ \
  backup@10.20.0.70:/srv/backups/ns-awx01/
```

4.2 — Off-site to AWS S3

`aws s3 sync` uploads only new or changed objects. The bucket has versioning enabled so an overwrite or deletion can be rolled back, and access is scoped to a narrow IAM policy that can write backups but not read unrelated buckets.

```
aws s3 sync /srv/backups/ns-awx01/ \
  s3://northstar-backups/ns-awx01/ \
  --storage-class STANDARD_IA --only-show-errors
```

4.3 — Lifecycle & retention policy

Keeping every backup in hot storage forever is wasteful. An S3 lifecycle policy automatically ages objects into cheaper tiers and expires them, while local copies are pruned by the script’s `find -mtime` step.

```
{
  "Rules": [{
    "ID": "db-backup-lifecycle",
    "Filter": { "Prefix": "" },
    "Status": "Enabled",
    "Transitions": [
      { "Days": 30, "StorageClass": "GLACIER" }
    ],
    "Expiration": { "Days": 365 },
    "NoncurrentVersionExpiration": { "NoncurrentDays": 90 }
  }]
}
```

Tier	Location	Retention
Hot / fast restore	Source host (local)	14 days, then pruned
On-site replica	Storage server (rsync)	30 days
Off-site standard	S3 STANDARD_IA	0–30 days
Off-site cold	S3 GLACIER	30–365 days, then expired

5 — Scheduling, Verification & Alerts

5.1 — Cron schedule

Each host runs its backup in a staggered window overnight via a `/etc/cron.d` entry, so dumps don't all hit the storage server and S3 at once. Output is appended to a log for auditing.

```
# /etc/cron.d/db-backups (m h dom mon dow user cmd)
0 2 * * * backup /opt/backup/db-backup.sh >> /var/log/db-backup.log 2>&1

# weekly restore-verification job
0 4 * * 0 backup /opt/backup/verify-restore.sh >> /var/log/verify.log 2>&1
```

5.2 — Restore verification

A backup is only real if it restores. A weekly job pulls the latest dump, verifies its checksum, and restores it into a throwaway scratch database — proving the artifact is usable, not just present.

```
#!/usr/bin/env bash
set -euo pipefail
LATEST=$(ls -t /srv/backups/ns-awx01/*.dump | head -1)

sha256sum -c "${LATEST}.sha256" # integrity gate
createdb verify_scratch
pg_restore -d verify_scratch "$LATEST" # prove it restores
dropdb verify_scratch
echo "restore verification OK: $LATEST"
```

5.3 — Slack notifications on failure

Failures are pushed to a Slack channel through an incoming webhook. The team learns about a broken backup the same morning, instead of discovering it during a restore that doesn't work. Notifications fire only on failure to avoid alert fatigue.

```
notify_fail() {
  curl -sf -X POST -H 'Content-type: application/json' \
    --data "{\"text\":\":x: Backup FAILED on $(hostname) — $1\"}" \
    "$SLACK_WEBHOOK"
}
# wired to the script's ERR trap so ANY failing step alerts:
trap 'notify_fail "step near line $LINENO"' ERR
```

6 — Outcomes

Before	After
Backups taken manually and inconsistently, if at all.	Every database dumped automatically on a staggered nightly cron schedule.
A single copy on the same host — one disk failure loses everything.	Three copies across two media with one off-site, per the 3-2-1 rule.
No way to know a dump was valid until a restore failed.	SHA-256 checksums plus a weekly restore test prove backups are usable.
Old backups piled up and consumed expensive storage indefinitely.	Retention pruning + S3 lifecycle age data to Glacier and expire it on schedule.
A failed backup went unnoticed for days or weeks.	Any failure in the chain raises an immediate Slack alert.

Technical stack

Layer	Technology
Database engines	PostgreSQL (pg_dump / pg_restore), MySQL (mysqldump)
Scripting	Bash (set -euo pipefail, ERR trap)
Integrity	SHA-256 checksums + weekly restore verification
On-site replication	rsync over SSH (incremental)
Off-site storage	AWS S3 (versioned bucket, aws s3 sync)
Cost / retention	S3 lifecycle (STANDARD_IA -> GLACIER -> expire)
Scheduling	cron (/etc/cron.d, staggered windows)
Alerting	Slack incoming webhook (failure-only)
OS	Linux (Rocky)

Names, hostnames, bucket names, and addresses in this document are anonymized for portfolio use. The architecture, tooling, and workflow reflect a real production deployment.